Weekly synthesis: whole-year parsing

5-11 October 2026 | C003a whole-year parsing | Written October 11; integrated October 12

Sidiq review status: Not yet reviewed. Synthetic engineering evidence only.

Findings and publication state

The parser now validates original whole-year text before calculating savings. Independent Python checks agree on 75 selected inputs, and deliberate faults show why successful conversion alone is insufficient. This is evidence about a bounded input contract, not investment performance or mastery. C003a consolidation remains unfinished. The complete CLI, vectors and statistics have not been built.

Five programme commits followed last Sunday's synthesis 9052698. All are on remote main, ending at 0a51f14e5bf88571438c6444a5f83d20a565ed7f. This afternoon's fresh fetch and independent remote lookup confirm equality; no unpushed commits exist. GitHub reports zero open issues and successful CI run 38120343379 for that exact head. The weekly synthesis adds no executable implementation.

October 11's draw of 1 was used by the morning failure-lesson integration. This synthesis and the separate reproduction lesson were saved pending. On October 12, all original input/artifact hashes matched before integration. Reproduction was integrated first; this report and the selectively merged backlog are the second coherent unit within October 12's draw of 2. Original observations remain dated.

Actual study and commit record

Study date	Completed evidence	Publication
October 5	Text versus value; safe fixed 2.5-to-2 counterexample	28c54a9, October 5
October 6	Transport interruption; no saved lesson or PDF	No commit; draw 0
October 7	Recovered grammar/range slot; prefix and signed-conversion examples	c77c6af, October 7
October 8	Whole-year parser; 30 C++ checks and fixed demonstration	57da84b, October 8
October 9	Independent 75-input oracle; two detected scratch mutations	093b2c4, October 9
October 10	Fractional-input failure lesson; five fixed cases	0a51f14, October 11
October 11 morning	Fresh temporary-build reproduction and teach-back material	Studied locally; integrated October 12

Draws for October 5-11 were 1, 0, 2, 3, 1, 0, 1. Actual committer-date counts were 1, 0, 1, 1, 1, 0, 1: five commits against eight available slots. Unused slots were not filled with artificial changes. Study dates and publication dates remain distinct. The October 6 failure shifted dependencies rather than causing extra lessons. Existing notes and completed evidence retain their original dates.

One concept: validate meaning before discarding evidence

Text preserves the supplied characters. **Parsing** interprets them. **Validation** checks the project's rules. **Conversion** changes a representation and can lose information. A correct calculation can answer the wrong question if its caller first changes the input.

The whole-year contract accepts a nonempty string of ASCII decimal digits representing a value from zero through the target int maximum, inclusive. Leading zeros are allowed. Signs, whitespace, decimal points, exponents, suffixes, non-ASCII bytes and embedded NUL are rejected. These are explicit project choices, not universal definitions of valid numbers. Error precedence is spelling before range.

Original input	Contract outcome	What it teaches
002	Return integer 2	Spelling may differ while value is equal
0	Return integer 0	Zero is valid evidence, not missing input
Blank	Reject	No supplied value is not zero years
2.5 or 2.0	Reject	Numeric conversion does not define the allowed grammar
2x or -1	Reject	Library success can violate whole-input or domain rules
2147483647	Accept on the tested target	Upper representation boundary is included
2147483648	Range error on the tested target	Form the successor in Python, not overflowing C++ int

The function scans the complete string, then uses checked base-ten std::from_chars conversion. It checks both the error code and stop position before returning. An exception supplies no normal year value; initialization to zero is not a fallback. The caller must preserve the full input length. The post-scan defensive conversion branch has no separately observed failure path.

Worked connection to the financial model

For whole n, fixed annual compounding is B=P*(1+r)^n. Here P=1000 arbitrary currency units, r=0.05 per model year and text 002 supplies n=2. Year one adds 50, reaching 1050; year two adds 52.50, reaching 1102.50. Independent Decimal interest credits agree with the displayed C++ answer.

Converting the separate numeric value 2.5 to int gives 2 in the fixed safe example. The savings function can calculate 1102.50 correctly for two years but cannot recover the discarded fraction. No fractional-year convention is defined, so the report does not invent a 2.5-year balance or loss. Microsoft's conversion reference supports truncation toward zero within the representable range; its from_chars reference documents error codes and stopping position. Sources are on the final page.

An accepted year count still does not guarantee a finite financial result: C002 separately rejects balance overflow. Work grows with input length, and external length/resource protection is unfinished. The contract assumes an ASCII-compatible execution character set on the tested targets.

Important results, negative controls and quality evidence

The 30 parser checks comprise eight accepted values, 21 required errors and one hand-calculated savings connection. The independent Python reference uses a complete byte-pattern match and Python integer arithmetic; it does not call the C++ conversion or obtain expected values from its output. Its 75 cases yield 47 acceptances, 21 spelling errors and seven range errors with no mismatches. It shares the written contract and trusts the target-limit report; agreement cannot prove that contract suits another application. Finite selected cases are not all-input correctness.

October 9's scratch exclusive-limit mutation wrongly rejects the maximum and is detected. Its NUL-truncation mutation wrongly turns the three-byte 2/NUL/x input into 2 and is also detected. These are recorded daily results, not mutation runs repeated by this weekly review.

October 10's correct pipeline accepts 002 and 0 and rejects 2.5, 0.5 and 2.0 before savings. A separate deliberately wrong adapter fully converts floating text, applies a safe finite [0,10] fixture guard, then casts. It produces exactly three forbidden acceptances and exits 1. The verifier requires the intended bytes, diagnostics and status; an arbitrary crash is not a successful control.

Fresh weekly checks versus recorded daily evidence

Evidence	Result and scope
Weekly make check	608 Python tests pass; no increase from October 4
Weekly Python quality	Ruff lint; 51 formatted files; strict mypy on 21 package files pass
Weekly C++ checks	66 savings plus 30 parser checks pass; four exact output fixtures pass
Weekly integrated experiments	75 oracle comparisons pass; correct/wrong fractional outputs and status pass
Weekly dependency check	pip reports no broken requirements
Weekly build provenance	Existing binaries reused; no new fresh-compilation claim
Morning integration record	Nine executables freshly compiled with C++17 and warnings as errors
Morning pending reproduction	Six fresh temporary executables; each evidence path run twice
Pending integrity audit	All 17 morning input hashes and five artifact hashes match

The morning reproduction used the same installed environment, not another provisioned machine. Its oracle outputs equal the archive; Decimal credits are 50.00 and 52.50. The weekly review reads and hash-verifies this evidence without repeating that lesson. Existing Python regression checks continue to protect earlier availability, calendar and return-accounting work.

No line/branch coverage percentage, mutation score, runtime speedup, security certification, empirical alpha or independent personal reproduction was measured. The successful remote CI belongs to published 0a51f14, not to today's ignored report. Passing checks do not change personal review status.

Failures and what changed because of them

October 6 stopped on a transport error before saving its teaching deliverables. October 7 recovered that prerequisite. The missed day remains visible; the foundation sequence was not compressed. Several daily PDFs initially had short overflow pages. Their validation records describe shortening repeated prose, rerendering and rechecking final text and every page. October 9's font-cache warnings were resolved with a writable font configuration. None of those initial versions is treated as final.

Restricted GitHub DNS failed during this weekly run. The approved network retry succeeded; no platform rejection or permission bypass occurred. The Jane Street source URL returned an application page without the cited role-description text, so this run does not refresh its requirements claim. Implementation checks in the inspected daily records passed; expected wrong-adapter failures were intentional observations, not unreported regressions. No additional weekly implementation was attempted.

Employer-skill connections, with limits

The dated role analysis covers 49 selected postings from eight employers, accessed September 21. It mixes junior and senior roles and counts explicit mentions, not universal hiring requirements. Python appears in 37, statistics and communication in 29 each, and C++ in 13. These historical counts have not been refreshed and say nothing about current vacancy or applicant eligibility.

Dated employer anchor	This week's demonstrable practice	Remaining gap
AQR data science: validation, Git, PyTest, CI/CD	Preserved fixtures, regression suite and exact-head CI evidence	No real-data model or production deployment
Jane Street research engineering: reproducible research code	Fixed inputs, recorded source hashes and fresh-build recipe	Same environment only; no scalable research platform
IMC C++ engineering	Explicit contracts, type/range boundaries and warning-clean builds	No profiling, concurrency or latency evidence
Communication across the sample	Worked examples, separate answers and honest negative findings	Sidiq teach-back remains unconfirmed

These are editorial links between small exercises and the archived role study, not employer endorsement or evidence that professional requirements are met. Source URLs and the local study are on the final page. C++ remains the default for new teaching projects; Python supplies independent verification.

Economic assumptions remain explicit

Savings assumes a fixed effective annual rate, reinvestment, no external cash flows, and no fees, tax or inflation. Displaying two decimals does not provide exact-money arithmetic. Costs are omitted, not estimated at zero from observations. No trades, market sample, tuning or return forecast exists. Q004a entitlement/sample evidence and Q005b provenance/integration still gate empirical work. Membership vintages, complete terminal proceeds, return basis and settlement distinctions remain unresolved inputs. The next ledger lesson must declare units and fees before reporting any accounting result.

Revised backlog: finish the prerequisite, narrow the ledger

The backlog revision is integrated October 12, preserving the published reproduction status. Failure and reproduction are integrated; consolidation remains the next learning step at integration.

Proposed date	One bounded focus	Evidence needed before marking done
October 12	C003a consolidation and readiness	Trace blank versus zero, suffix, fraction and range errors; record unresolved prerequisites
October 13	L001a: cash, position and marked wealth	One hand ledger with explicit units; no new C++ abstraction
October 14	L001b: derive one buy and fee	Cash' = cash - quantity*price - fee; hand conservation check
October 15	L001c: introduce one position struct	Explain each field; fixed initial state and known marked value
October 16	L001d: one validated buy function	Fixed synthetic buy; reject invalid values and unaffordable orders without state change
October 17	L001e: boundaries and independent oracle	Exact hand/Python reference; zero/invalid/cost cases; no price signal
October 18	L001f: reproduce and consolidate	Reconcile cash plus holdings; record unresolved limits; weekly review separate

Dates are conditional. Repeat unclear prerequisites and shift later slots without adding lessons. Personal review remains unconfirmed; ordinary authorised work can continue.

Split the broad ledger goal into L001 fixed-event accounting and L002 delayed decision/execution. L002 follows L001 and introduces event timing, then the test that changing future prices cannot change past decisions. A one-buy example cannot satisfy that test by itself. Trend implementation cannot advance until its accounting, timing and return prerequisites pass; October 19 is conditional. This makes any postponed advanced extension explicit instead of compressing the strategy calendar.

C003b remains deferred with principal/rate grammar, finite-value, units and resource checks outstanding. The fixed-input ledger needs no full CLI. C004 vectors/returns and C005 statistics remain carryover, introduced before projects needing them. Q011/Q012/Q013 retain their order; Q004a/Q005b still gate empirical work. Dates do not complete prerequisites or authorise new scope.

Questions for Sidiq and priorities for the next run

Which parser step needs repetition: text versus value, exception flow or range boundaries? The revised ledger pace follows the curriculum's repeat-and-simplify rule; any preference to spend longer on foundations can steer it. No answer is needed to continue bounded synthetic work.

Before real data: identify existing lawful access or a source/budget preference, without sending credentials. Any narrower historical-vintage claim requires explicit agreement; none is assumed. The separate finance project's identity remains unresolved and is outside this week's implementation.

Revision questions

1. Why is returning zero on invalid text different from accepting the original string 0?

2. Why can std::from_chars success on 2x or -1 still violate this parser's contract?

3. What does agreement on 75 inputs establish, and what does it leave uncertain?

4. Why must the wrong adapter produce the expected mismatch count and exit status, rather than any failure?

5. What remains unfinished before a ledger can support a claim about delayed trading decisions?

Answers - read after attempting the questions

1. Zero is an explicit valid count with B(0)=P. Invalid text supplies no count; substituting zero hides failure and manufactures a calculation.

2. 2x can yield a valid numeric prefix while leaving a suffix. -1 can be fully consumed but violates the project's digit-only, nonnegative spelling rule. Library conversion and domain validation differ.

3. Exact values and error categories agree for selected finite cases using different conversion implementations. The methods share a contract and target-limit input. They do not prove exhaustive correctness, suitable economic assumptions, coverage or mastery.

4. A crash, stale binary or unrelated error could fail without detecting the intended defect. Exact observables tie rejection to the three forbidden fractional spellings in the fixed fixture.

5. C003a consolidation comes first. Then fixed-event cash/position accounting and its independent checks; a later decision/execution timing slice must prove future-price invariance before trading claims.

Evidence and primary references

Repository evidence: research_log/2026-10-05.md, 2026-10-07.md, 2026-10-08.md, 2026-10-09.md and 2026-10-10.md, with corresponding reports/milestone validation records. Morning evidence: research_log/2026-10-11.md, reports/milestone/2026-10-11-validation.md and experiments/years_parser/reproduction-2026-10-11.json; original pending hashes remain preserved. Experiments: experiments/years_parser/. Role study: career_research/role_skill_analysis.md and role_skill_matrix.csv. The dated integration JSON beside this report preserves original source and artifact hashes.

Rechecked October 11: Microsoft Standard conversions, fractional truncation and representability: https://learn.microsoft.com/en-us/cpp/cpp/standard-conversions?view=msvc-170

Rechecked October 11: Microsoft charconv functions, error code and stop-position semantics: https://learn.microsoft.com/en-us/cpp/standard-library/charconv-functions?view=msvc-170

Dated employer anchors from September 21, not refreshed requirement claims:
https://careers.aqr.com/jobs?gh_jid=8088184
https://www.janestreet.com/join-jane-street/apply/7437779002
https://job-boards.eu.greenhouse.io/imc/jobs/4945420101

Current published-head CI:
https://github.com/SidiqB/quant-research-apprenticeship/actions/runs/38120343379

The weekly schedule remains Sunday 16:00 Europe/London, following daylight-saving changes. Programme work stops after December 31, 2026, pending the next scope. The next run must use the shared lock, reuse its London-date draw, count actual commits, and integrate pending work only within the allowance.